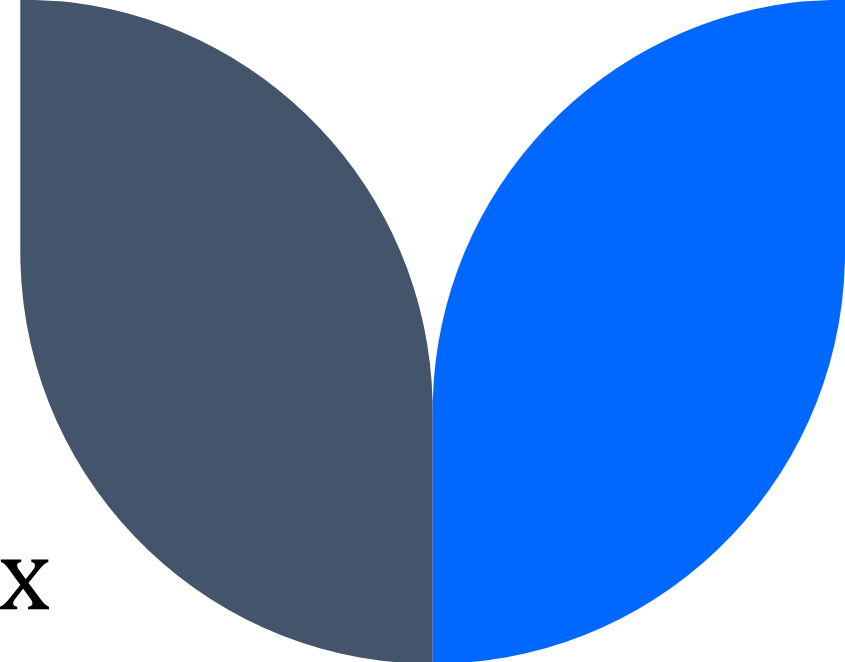
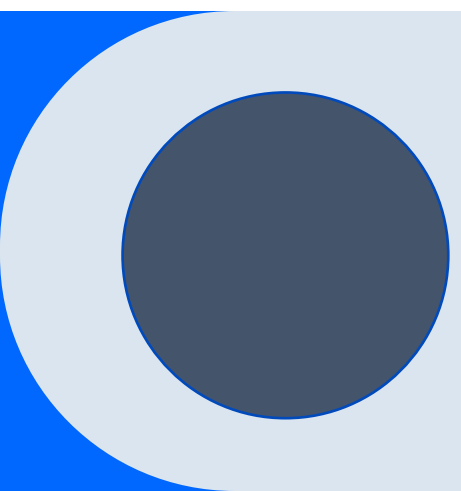




12 правил Кодда. Использование табличных выражений. Множества группировок и поворот данных

Валова Анастасия Александровна
НИТУ «МИСиС»



Термины описания структуры таблиц

Формальный термин	Неформальный эквивалент
Отношение	Таблица
Кортеж	Строка или запись
Кардинальность	Количество строк
Атрибут	Столбец или поле
Степень отношения	Количество столбцов
Первичный ключ	Уникальный идентификатор
Домен	Множество допустимых значений

12 правил Кодда (реляционные СУБД)

0. Основное правило (*Foundation Rule*)

Реляционная система управления базами данных (СУБД), должна быть способна управлять БД, используя исключительно свои реляционные возможности.

1. Информационное правило (*The Information Rule*)

Вся информация в реляционной базе данных на логическом уровне должна быть явно представлена единственным способом: значениями в таблицах.

2. Гарантированный доступ к данным (*Guaranteed Access Rule*)

В реляционной БД каждое отдельное (атомарное) значение данных должно быть логически доступно с помощью комбинации имени таблицы, значения первичного ключа и имени столбца.



12 правил Кодда (реляционные СУБД)

3. Систематическая поддержка NULL (Systematic Treatment of Null Values)

Неизвестные, или отсутствующие значения **NULL**, отличные от любого известного значения, должны поддерживаться для всех типов данных при выполнении любых операций. NULL не сводится к 0 для арифметических типов или к пустым строкам для символьных типов.

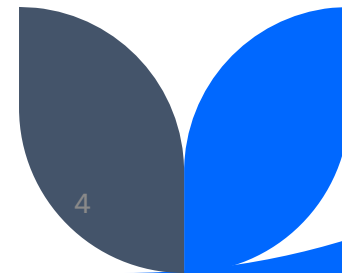
4. Доступ к каталогу данных в реляционной модели (Active On-Line Catalog Based on the Relational Model)

Схема БД должна представляться в форме реляционных таблиц с возможностью доступа к ним с использованием реляционной модели (по аналогии с пользовательскими таблицами).

5. Полнота подмножества языка (Comprehensive Data Sublanguage Rule)

Реляционная СУБД должна поддерживать хотя бы один реляционный язык, который:

- (а) имеет линейный синтаксис,
- (б) может использоваться как интерактивно, так и в прикладных программах,
- (в) поддерживает операции определения структур данных, определения представлений, манипулирования данными, ограничения целостности, управления правами доступа и операции управления транзакциями.



12 правил Кодда (реляционные СУБД)

6. **Возможность изменения представлений** (View Updating Rule)

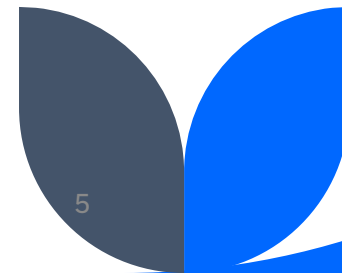
Каждое представление должно поддерживать все операции манипулирования данными, которые поддерживают реляционные таблицы: операции выборки, вставки, изменения и удаления данных.

7. **Наличие высокоуровневых операций управления данными** (High-Level Insert, Update, and Delete)

Операции вставки, изменения и удаления данных должны поддерживаться не только по отношению к одной строке реляционной таблицы, но и по отношению к любому множеству строк.

8. **Физическая независимость данных** (Physical Data Independence)

Приложения не должны зависеть от используемых способов хранения данных на носителях, от аппаратного обеспечения компьютеров, на которых находится реляционная БД.



12 правил Кодда (реляционные СУБД)

9. Логическая независимость данных (Logical Data Independence)

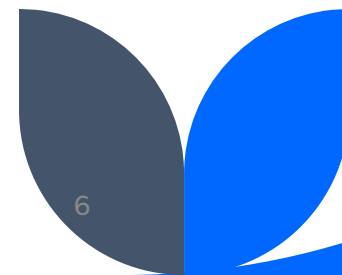
Представление данных в приложении не должно зависеть от структуры реляционных таблиц. Если в процессе нормализации одна реляционная таблица разделяется на две, представление должно обеспечить такое соединение этих данных, чтобы изменение структуры реляционных таблиц не сказывалось на работе приложений.

10. Независимость контроля целостности (Integrity Independence)

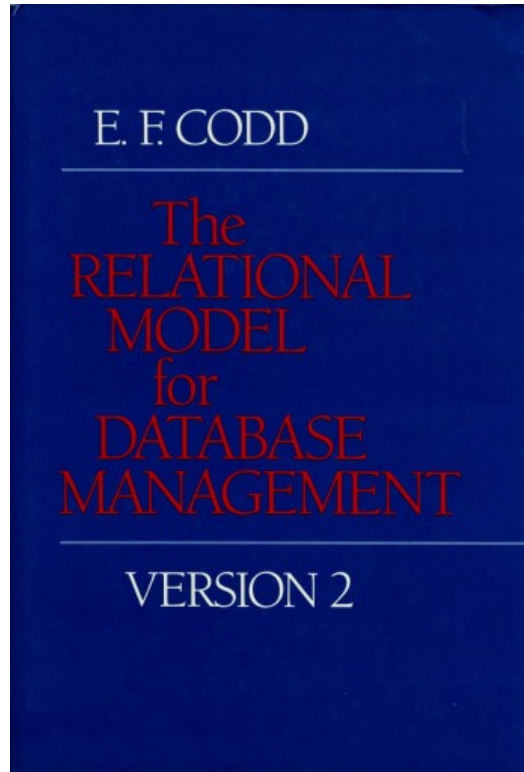
Вся информация, необходимая для поддержки целостности, должна находиться в каталоге данных (см. правило 4). Язык для работы с данными должен выполнять проверку входных данных и автоматически поддерживать целостность данных.

11. Независимость от расположения (Distribution Independence)

База данных может быть распределённой, может находиться на нескольких компьютерах, и это не должно оказывать влияния на приложения. Перенос базы данных на другой компьютер не должен оказывать влияния на приложения.



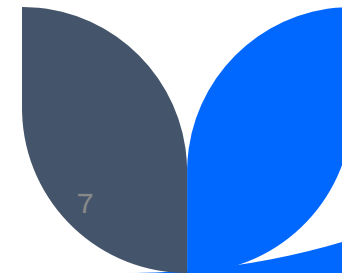
12 правил Кодда (реляционные СУБД)



Codd, Edgar F.
The relational model for database
management: Version 2.
Addison-Wesley, 1990.
ISBN 9780201141924.

12. Согласование языковых уровней (The Nonsubversion Rule)

Если используется низкоуровневый язык доступа к данным, он не должен игнорировать правила безопасности и правила целостности, которые поддерживаются языком более высокого уровня.



7 Использование табличных выражений

Что такое представление (View)?

- Объект БД, на который можно ссылаться, как на таблицу.
 - CREATE VIEW название_представления AS выражение_SELECT
- Именованный SELECT-запрос:

```
CREATE VIEW HumanResources.EmployeeList  
  (EmployeeID, FamilyName, GivenName)  
  AS  
  SELECT EmployeeID, LastName, FirstName  
  FROM HumanResources.Employee;
```

Введение в представления

- Представление не сохраняет данные (кроме индексируемых).
- **WITH SCHEMABINDING** блокирует изменения в схеме базовых таблиц.
- Добавление уникального кластерного индекса делает представление индексируемым:
 - Данные сохраняются на диск, увеличивая производительность
- SQL Server Enterprise Edition учитывает индексируемые представления.
- Вставка и обновление данных в представление может происходить только в одной базовой таблице.

Запросы к представлениям

- Представления (отображения) – именованные запросы с хранимым в БД определением:
 - Обеспечивают абстракцию, инкапсуляцию и упрощение;
 - С точки зрения администратора, отображения обеспечивают дополнительный уровень безопасности.
- Используются в запросах SELECT как таблицы:

```
CREATE VIEW Sales.vSalesOrders
AS
SELECT  oh.OrderID, oh.Orderdate, oh.CustomerID,
        od.ProductID, od.Quantity
FROM    Sales.OrderHeaders AS oh
INNER JOIN Sales.OrderDetails AS od
ON      od.OrderID = oh.OrderID;
```

```
SELECT OrderID, CustomerID, ProductID
FROM Sales.vSalesOrder
ORDER BY OrderID;
```

Временные таблицы

- Временные таблицы предназначены для хранения временных результатов **в рамках сессии** пользователя:
 - Создаются в БД tempdb и удаляются автоматически;
 - Создаются с префиксом “#”;
 - Глобальные временные таблицы создаются с префиксом “##”.

```
CREATE TABLE #tmpProducts  
(ProductID INTEGER,  
 ProductName varchar(50));  
GO  
...  
SELECT * FROM #tmpProducts;
```

Табличные переменные

- Введены из-за частых рекомпиляций временных таблиц;
- Используются как временные таблицы, но видимость ограничена текущим пакетом;
- Не имеют статистики, поэтому изменения не вызывают рекомпиляции (всегда ожидается 1 строка);
- Используйте только на малых наборах данных.

```
DECLARE @varProducts table  
(ProductID INTEGER,  
  ProductName varchar(50));  
...  
SELECT * FROM @varProducts
```

Табличные функции

- TF – именованные объекты с определениями хранимыми в БД;
- TF возвращают виртуальную таблицу в точку вызова;
- В отличие от представлений, TF поддерживают входные параметры:
 - TF можно определить, как параметризованные представления.

```
CREATE FUNCTION Sales.fn_GetOrderItems (@OrderID AS Integer)
RETURNS TABLE
AS
RETURN
(SELECT ProductID, UnitPrice, Quantity
FROM Sales.OrderDetails
WHERE OrderID = @OrderID);
...
SELECT * FROM Sales.fn_GetOrderItems (1025) AS LineItems;
```

Основы пользовательской табличной функции

- Предложение RETURNS также определяет формат таблицы. Область видимости табличной переменной локализована внутри функции.
- Операторы INSERT Transact-SQL в теле функции формируют ответ, добавляя строки в возвращаемую переменную, определенную в предложении RETURN.
- При выполнении оператора RETURN данные из табличной переменной возвращаются в виде результирующей таблицы. Оператор RETURN не может иметь аргументов.

Производные таблицы (Derived Tables)

Введение

- Производные таблицы – именованные запросы, созданные в рамках внешнего оператора SELECT.
- Не хранятся в БД – представляют виртуальную таблицу.
- Область видимости производной таблицы – запрос, в котором она определена.

```
SELECT orderyear, COUNT(DISTINCT custid) AS cust_count  
FROM  
    (SELECT YEAR(orderdate) AS orderyear, custid  
     FROM Sales.Orders) AS derived_year  
GROUP BY orderyear;
```


Производные таблицы.

Основные правила

- Производные таблицы **обязаны**:
 - Иметь псевдоним;
 - Иметь уникальные имена для всех столбцов;
 - Не использовать предложение ORDER BY clause (без TOP или OFFSET/FETCH);
 - Избегать множественных ссылок в том же запросе.
- Производные таблицы **могут**:
 - Использовать внутренние или внешние псевдонимы для столбцов;
 - Ссылаться на параметры и/или переменные;
 - Быть вложенными в другие производные таблицы.

Производные таблицы

Указание псевдонимов столбцов: 2 варианта

- Псевдонимы столбцов могут объявляться внутри:

```
SELECT orderyear, COUNT(DISTINCT custid) AS cust_count  
FROM ( SELECT YEAR(orderdate) AS orderyear, custid  
        FROM Sales.Orders) AS derived_year  
GROUP BY orderyear;
```

- Или снаружи:

```
SELECT orderyear, COUNT(DISTINCT custid) AS cust_count  
FROM ( SELECT YEAR(orderdate), custid  
        FROM Sales.Orders) AS  
        derived_year(orderyear, custid)  
GROUP BY orderyear;
```

Обобщенные табличные выражения (СТЕ)

- СТЕ – именованные табличные выражения, определенные в запросе;
- СТЕ похожи на производные таблицы по области видимости и требованиям к именованию;
- В отличие от производных таблиц, СТЕ поддерживают множественные ссылки и рекурсию.

```
WITH CTE_year (OrderYear, CustID)
AS
(
    SELECT YEAR(orderdate), custid
    FROM Sales.Orders
)
SELECT OrderYear, COUNT(DISTINCT CustID) AS Cust_Count
FROM CTE_year
GROUP BY orderyear;
```

Обобщенные табличные выражения (СТЕ)

Рекурсия

- Укажите корневой запрос;
- Используйте UNION ALL, чтобы добавить рекурсивный запрос;
- Запросы к СТЕ могут использовать опцию MAXRECURSION.

```
WITH OrgReport (ManagerID, EmployeeID, EmployeeName, Level)
AS
(
    SELECT e.ManagerID, e.EmployeeID, EmployeeName, 0
    FROM HR.Employee AS e
    WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.ManagerID, e.EmployeeID, e.EmployeeName, Level + 1
    FROM HR.Employee AS e
    INNER JOIN OrgReport AS o ON e.ManagerID = o.EmployeeID
)
SELECT * FROM OrgReport
OPTION (MAXRECURSION 3);
```

8 Множества группировок и поворот данных

Множества группировок

Синтаксис

- Раздел GROUPING SETS основан на предложении GROUP BY;
- Позволяет задавать множество группировок в одном запросе.

```
SELECT <список столбцов с агрегатами>
FROM <источник>
GROUP BY
GROUPING SETS
(
    <column_name>, -- один или более столбцов
    <column_name>, -- один или более столбцов
    () -- круглые скобки, если надо агрегировать все строки
);
```

Множества группировок

Пример

	EmployeeID	CustomerID	TotalAmount
Итого для всех продаж	NULL	NULL	256.23
Подытог для каждого клиента	NULL	1	49.99
	NULL	2	107.49
	NULL	3	98.75
Подытог для каждого сотрудника	1	NULL	107.49
	2	NULL	148.74

```
SELECT EmployeeID, CustomerID, SUM(Amount) AS TotalAmount
FROM Sales.SalesOrder
GROUP BY
GROUPING SETS(EmployeeID, CustomerID,());
```

ROLLUP и CUBE

- ROLLUP обеспечивает множество группировок с комбинациями в иерархическом стиле:

```
SELECT StateProvince, City, COUNT(CustomerID) AS Customers  
FROM Sales.vCustomerDetails  
GROUP BY ROLLUP(StateProvince, City)  
ORDER BY StateProvince, City;
```

- CUBE обеспечивает множество группировок с перебором всех возможных комбинаций для группировки:

```
SELECT SalesPersonName, CustomerName, SUM(Amount) AS TotalAmount  
FROM Sales.vSalesOrders  
GROUP BY CUBE(SalesPersonName, CustomerName)  
ORDER BY SalesPersonName, CustomerName;
```


Идентификация группировок в результатах

- Несколько группировок затрудняют идентификацию источника каждой строки в результирующем наборе;
- NULL может появиться из данных или представлять значение-заполнитель в группировке;
- Через функцию GROUPING_ID можно определить группировку по столбцу (результат 1 или 0):

```
SELECT GROUPING_ID(SalesPersonName) AS SalesPersonGroup,  
       GROUPING_ID(CustomerName) AS CustomerGroup,  
       SalesPersonName, CustomerName, SUM(Amount) AS TotalAmount  
FROM Sales.vSalesOrders  
GROUP BY CUBE(SalesPersonName, CustomerName)  
ORDER BY SalesPersonName, CustomerName;
```

GROUPING_ID для нескольких столбцов

```
SELECT a, b, c, SUM(d), GROUPING_ID(a,b,c)
FROM T GROUP BY <список group by>
```

Статистически обработанные столбцы	Входные данные GROUPING_ID (a, b, c) = GROUPING(a) + GROUPING(b) + GROUPING(c)	Выходные данные GROUPING_ID ()
a	100	4
b	010	2
c	001	1
ab	110	6
ac	101	5
bc	011	3
abc	111	7

GROUPING_ID () возвращает битовое представление типа integer, в котором N младших битов могут быть установлены (1).

Установленный бит означает, что соответствующий аргумент не является столбцом группирования для указанной выходной строки.

Применительно к одиночному запросу группировки функция GROUPING (<выражение-столбец>) эквивалентна GROUPING_ID (<выражение-столбец>), и обе эти функции возвращают 0.

Поворот данных (PIVOT)

- Поворот данных – это транспонирование данных из построчной ориентации в ориентацию на основе столбцов;
- Уникальные значения из одного столбца проецируются в заголовки столбцов — может происходить агрегация:

OrderID	ProductID	Category	Revenue
1023	1	Bikes	1078.75
1023	15	Accessories	52.00
1023	21	Accessories	124.90
1024	1	Bikes	2491.00
1025	3	Bikes	1067.49
1025	15	Accessories	125.99
1025	35	Clothing	26.57
1025	36	Clothing	5.78

OrderID	Bikes	Accessories	Clothing
1023	1078.75	176.90	<i>NULL</i>
1024	2491.00	<i>NULL</i>	<i>NULL</i>
1025	1067.49	125.99	32.35



```
SELECT OrderID, Bikes, Accessories, Clothing
FROM
    (SELECT OrderID, Category, Revenue FROM Sales.SalesDetails) AS sales
PIVOT (SUM(Revenue) FOR Category IN([Bikes], [Accessories], [Clothing]))
AS pvt
```

Обратный поворот данных (UNPIVOT)

- Обратный поворот – это поворот данных из ориентации на основе столбцов в ориентацию на основе строк;
- Распространяет или разделяет значения из одной строки-источника в одну или несколько целевых строк;
- Каждая строка-источник становится одной или несколькими строками в результате, основанном на количестве поворотных столбцов:

OrderID	Bikes	Accessories	Clothing
1023	1078.75	176.90	NULL
1024	2491.00	NULL	NULL
1025	1067.49	125.99	32.35

OrderID	Category	Revenue
1023	Bikes	1078.75
1023	Accessories	176.90
1024	Bikes	2491.00
1025	Bikes	1067.49
1025	Accessories	125.99
1025	Clothing	32.35

SELECT OrderID, Category, Revenue
FROM

(SELECT OrderID, Bikes, Accessories, Clothing FROM Sales.SalesByCat) AS pvt
UNPIVOT (Revenue FOR Category IN([Bikes], [Accessories], [Clothing])) AS unpvt